Lab 5: Monster Identification

Scot Schmidl
sschmidl3@gatech.edu

1 HOW DOES YOUR AGENT WORK?

My agent treats monster identification as a concept classification voting problem over the twelve attributes. For each case, it first partitions the labeled examples into positive and negative sets. The core idea is to look at each attribute of the new monster and ask whether that value has appeared before and did it show up more often as a positive or negative example. That gives a simple voting scheme across attributes, i.e. if an attribute value has more positive matches than negative it casts a +1 vote, if it has more negative matches it casts a -1 vote, if it ties or has no matches it casts a 0 vote.

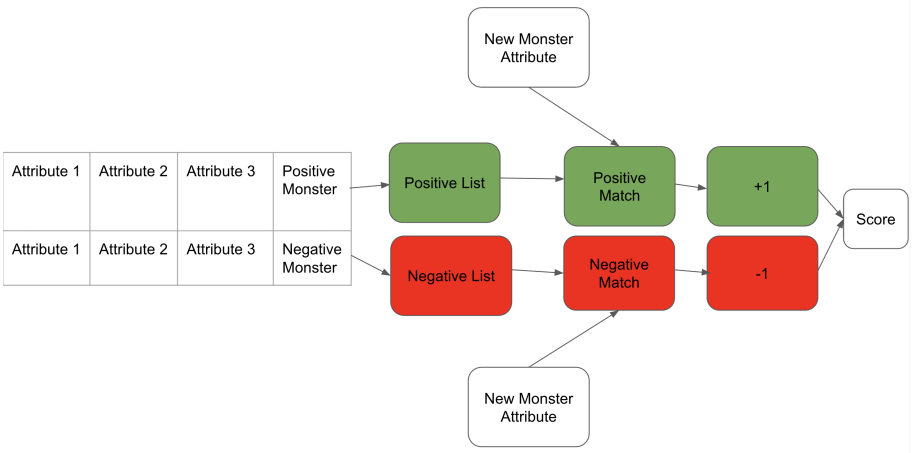


Figure 1—Attribute > List > New Monster > Matches > Voting > Score

This design is essentially a knowledge-based classifier that can resemble version space and concept learning, but implemented as a direct tally instead of maintaining general and specific boundaries. The algorithm assumes independence of attributes, i.e. each attribute contributes its own evidence, and the agent aggregates that evidence linearly rather than modeling interactions. Given that the domain is small and discrete, the learning is simply counting occurrences in the observed examples. It doesn't fit any continuous or probabilistic models.

2 HOW WELL DOES YOUR AGENT PERFORM?

Against the test cases, my agent correctly identifies 40 out of 40 monsters. In other words, whenever the labeled examples defined a particular monster species and the agent was presented with a new monster, the net vote of attribute values matched the ground truth label in all cases. This suggests that, within the constraints, the simple voting strategy is sufficient to reconstruct the species.

In terms of failures, the agent is most vulnerable to cases where the new monster has attributes that have only ever appeared in negative examples, but the overall concept is still positive. In those cases, the relevant attributes will cast -1 votes and if there are not enough other attributes with positive evidence to balance the negatives, the final score could be false. My implementation relies purely on frequency of attributes, which means it can struggle with very sparse or misleading samples even though that didn't show up in these cases.

3 HOW EFFICIENT IS YOUR AGENT?

The agent runs in linear time in the size of the training set. It scans the labeled examples once to build the positive and negative lists, then for each of the attributes on a new monster, it loops over those lists to count matches. If there are n labeled monsters, the worse case time complexity is $O(n * A)$, where A is the number of attributes. If A remains constant, the time complexity becomes O(n) per classification. Space usage is also linear in n, because the agent stores the positive and negative monsters in two lists without building any other data structures. Given the relatively small datasets, the cost is negligible, and the agent runs quickly even when there are many labeled monsters. If we were to scale up to much larger numbers of examples, the counting loops would grow proportionally, which is predictable but not optimized.

4 DOES YOUR AGENT DO ANYTHING CLEVER?

The main cleverness of the agent is how it turns the independence of attributes into a compact decision rule. Instead of learning a complex concept description with explicit ranges or logical constraints, it lets each attribute value vote based on support from the labeled examples, then uses the sign of the total score as the final decision. The tie-breaking rule just states, "when the evidence is ambiguous assume negative so exclusion". This matches the idea of making an educated guess in uncertain cases.

Another design choice is that the agent treats each example without weighting them even if some seem more likely to be a monster. This helps prevent injecting bias and overfitting. I would say that overall, this is a simple heuristic classifier rather than a deep reasoning system, but the voting system is similar to some case-based reasoning and incremental concept learning systems.

5 HOW DOES YOUR AGENT COMPARE TO A HUMAN?

As a human, I would solve these problems in a similar way. I would look at the positive examples, notice typical traits, and then count how many of those traits a new monster shares. Where the agent will differ is that it considers every attribute as one vote, whereas I might inject domain knowledge about what I think a monster looks like.

Humans could also be more flexible when new values show up. We can reason by analogy or by common sense, while my agent treats unseen values as neutral unless they have appeared in negative examples. On the other hand, the agent is consistent and doesn't change its mind based on framing or order of examples. Given the same labeled dataset, it will always compute the same score and produce the same result.